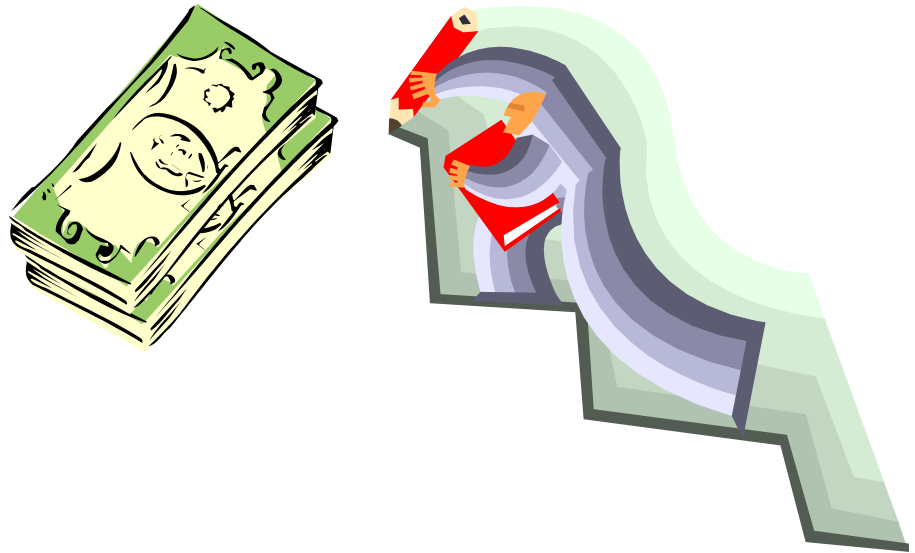


Lecture 09

Greedy Algorithms

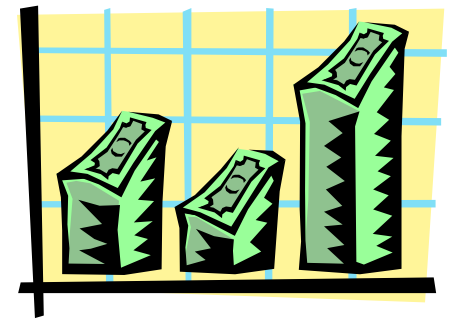


Outline and Reading



- ◆ The Greedy Method Technique (5.1)
- ◆ Fractional Knapsack Problem (5.1.1)
- ◆ Dijkstra's algorithm (7.1.1)
- ◆ Prim-Jarnik's Algorithm (7.3.2)
- ◆ Kruskal's Algorithm (7.3.1)

The Greedy Method Technique



- ◆ **The greedy method** is a general algorithm design paradigm, built on the following elements:
 - **configurations:** different choices, collections, or values to find
 - **objective function:** a score assigned to configurations, which we want to either maximize or minimize
- ◆ It works best when applied to problems with the **greedy-choice** property:
 - a globally-optimal solution can always be found by a series of local improvements from a starting configuration.

Making Change



- ◆ Problem: A dollar amount to reach and a collection of coin amounts to use to get there.
- ◆ Configuration: A dollar amount yet to return to a customer plus the coins already returned
- ◆ Objective function: Minimize number of coins returned.
- ◆ Greedy solution: Always return the largest coin you can
- ◆ Example 1: Coins are valued \$.32, \$.08, \$.01
 - Has the greedy-choice property, since no amount over \$.32 can be made with a minimum number of coins by omitting a \$.32 coin (similarly for amounts over \$.08, but under \$.32).
- ◆ Example 2: Coins are valued \$.30, \$.20, \$.05, \$.01
 - Does not have greedy-choice property, since \$.40 is best made with two \$.20's, but the greedy solution will pick three coins (which ones?)

The Fractional Knapsack Problem



- ◆ Given: A set S of n items, with each item i having
 - b_i - a positive benefit
 - w_i - a positive weight
- ◆ Goal: Choose items with maximum total benefit but with weight at most W .
- ◆ If we are allowed to take fractional amounts, then this is the **fractional knapsack problem**.
 - In this case, we let x_i denote the amount we take of item i
 - Objective: maximize
$$\sum_{i \in S} b_i (x_i / w_i)$$
 - Constraint:
$$\sum_{i \in S} x_i \leq W$$

Example



- ◆ Given: A set S of n items, with each item i having
 - b_i - a positive benefit
 - w_i - a positive weight
- ◆ Goal: Choose items with maximum total benefit but with weight at most W .

Items:					
	1	2	3	4	5
Weight:	4 ml	8 ml	2 ml	6 ml	1 ml
Benefit:	\$12	\$32	\$40	\$30	\$50



10 ml

“knapsack”

What item(s) to choose?

Example



- ◆ Objective function: Maximum total benefit
- ◆ Greedy solution: Always choose the item with largest value

Items:						 10 ml “knapsack” Solution: • 1 ml of item 5 • 2 ml of item 3 • 6 ml of item 4 • 1 ml of item 2 Max benefit = 124
Weight:	4 ml	8 ml	2 ml	6 ml	1 ml	
Benefit:	\$12	\$32	\$40	\$30	\$50	
Value:	3	4	20	5	50	
(\$ per ml)						

The Fractional Knapsack Algorithm



◆ Greedy choice: Keep taking item with **highest value** (benefit to weight ratio)

- Since $\sum_{i \in S} b_i (x_i / w_i) = \sum_{i \in S} (b_i / w_i) x_i$
- Run time: $O(n \log n)$. Why?

◆ Correctness: Suppose there is a better solution

- there is an item i with higher value than a chosen item j , but $x_i < w_i$, $x_j > 0$ and $v_i > v_j$
- If we substitute some j with i , we get a better solution
- How much of i : $\min\{w_i - x_i, x_j\}$
- Thus, there is no better solution than the greedy one

Algorithm fractionalKnapsack(S, W)

Input: set S of n items with benefit b and weight w ; max weight W

Output: amount x of each item i to maximize benefit with weight at most W

Q = empty maxheap priority queue with v as key

for each item i in S

$i.x = 0$

$i.v = i.b / i.w$ // value

$Q.insert(i)$

$w = 0$ // total weight

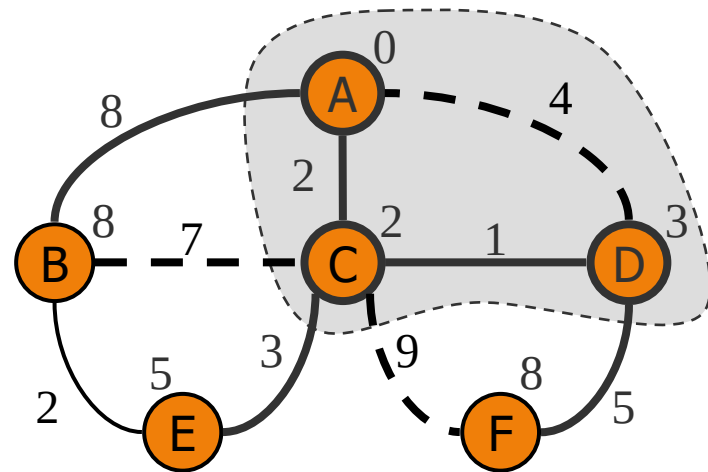
while $w < W$

$i = Q.removeMax()$ // highest v

$i.x = \min(i.w, W - w)$

$w = w + i.x$

Shortest Paths



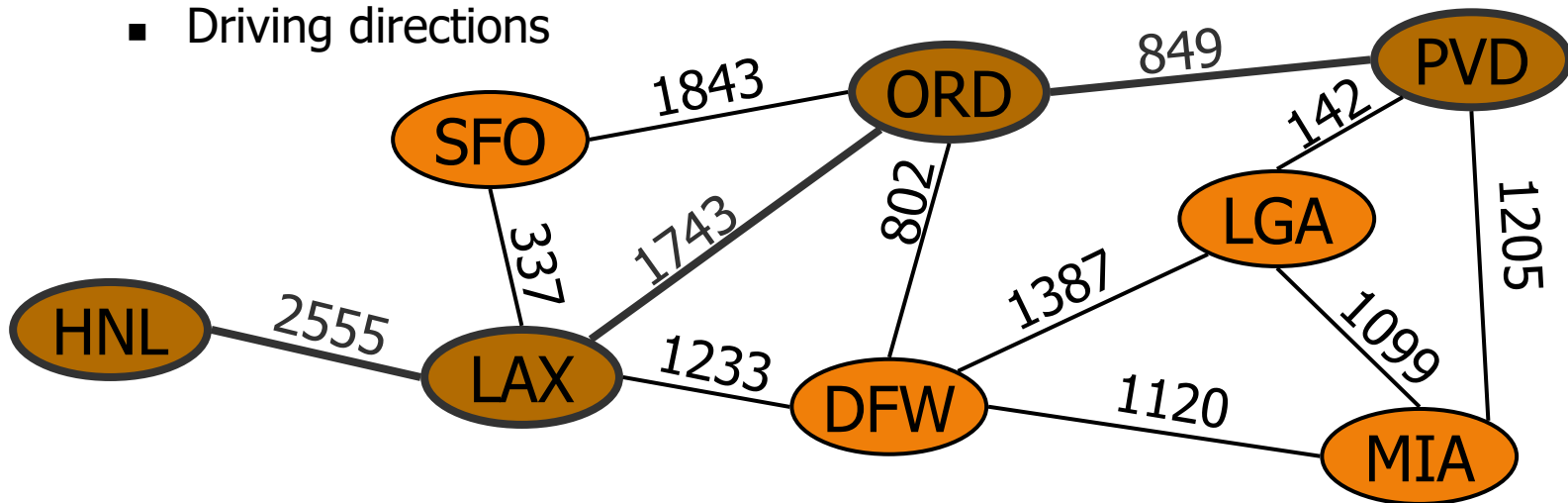
Outline and Reading

- ◆ Shortest path problem
 - Shortest path properties
- ◆ Dijkstra's algorithm
 - Algorithm
 - Edge relaxation

Shortest Path Problem



- ◆ Given a weighted graph and two vertices u and v , we want to find a path of minimum total weight between u and v .
 - Length of a path is the sum of the weights of its edges.
- ◆ Example:
 - Shortest path between Providence and Honolulu
- ◆ Applications
 - Internet packet routing
 - Flight reservations
 - Driving directions



Shortest Path Properties



Property 1:

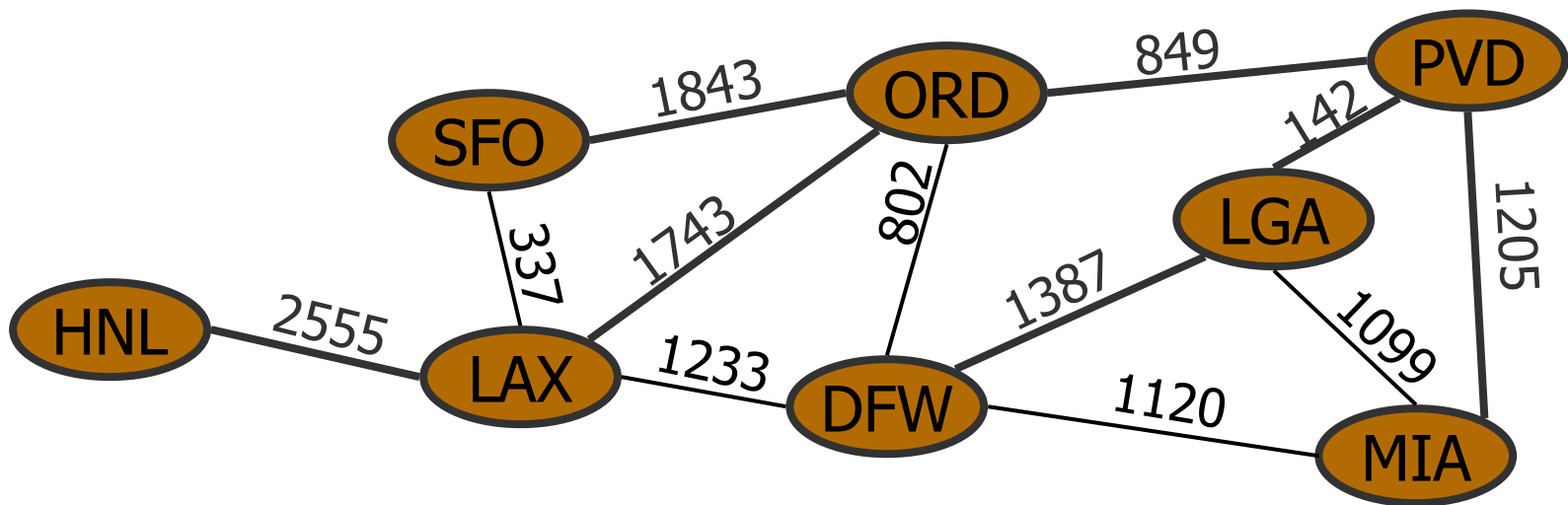
A subpath of a shortest path is itself a shortest path

Property 2:

There is a tree of shortest paths from a start vertex to all the other vertices

Example:

Tree of shortest paths from Providence





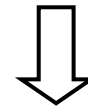
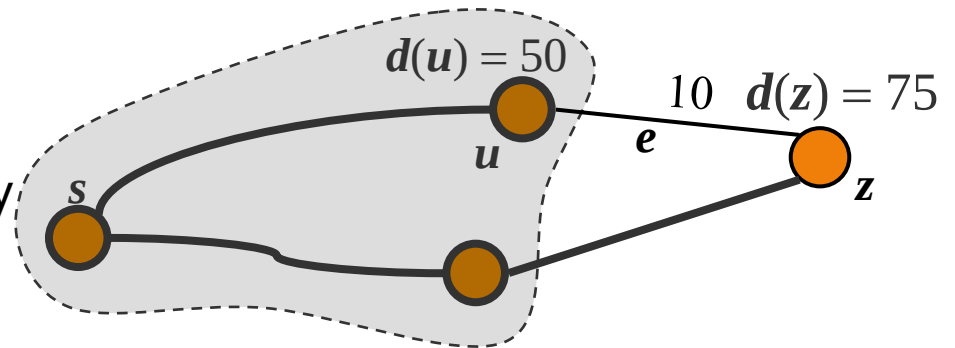
Dijkstra's Algorithm

- ◆ The distance of a vertex v from a vertex s is the length of a shortest path between s and v
- ◆ Dijkstra's algorithm computes the distances of all the vertices from a given start vertex s
- ◆ Assumptions:
 - the graph is connected
 - the edges are undirected
 - the edge weights are nonnegative
- ◆ We grow a “**cloud**” of vertices, beginning with s and eventually covering all the vertices
- ◆ We store with each vertex v a label $d(v)$ representing the distance of v from s in the subgraph consisting of the cloud and its adjacent vertices
- ◆ At each step
 - We add to the cloud the vertex u outside the cloud with the smallest distance label, $d(u)$
 - We update the labels of the vertices adjacent to u

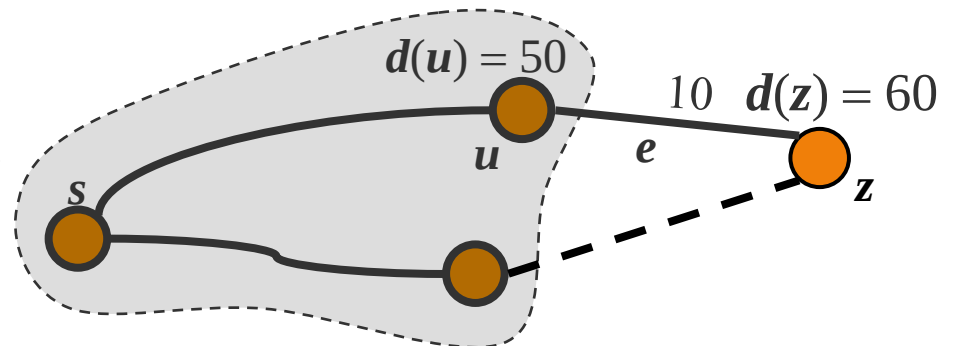
Edge Relaxation



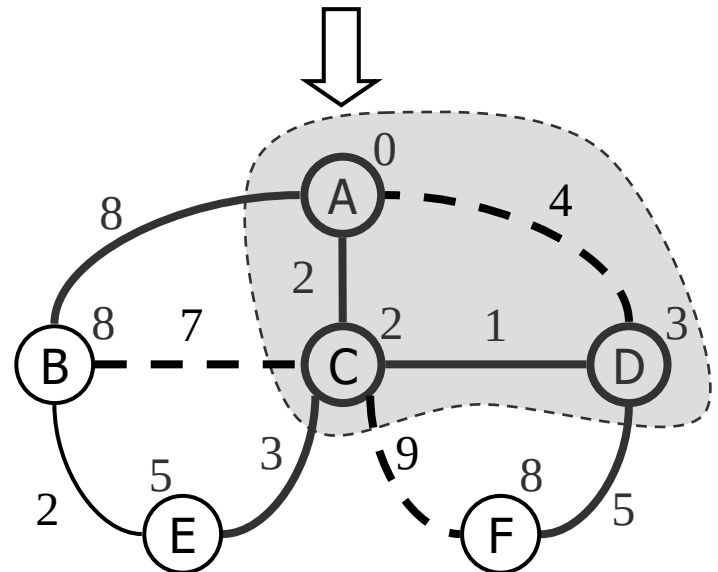
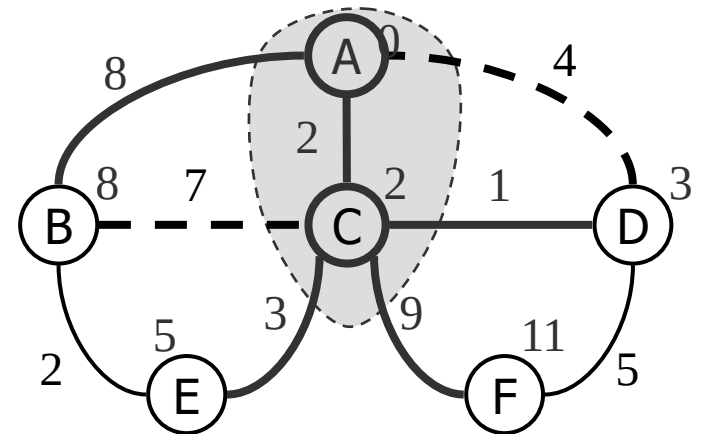
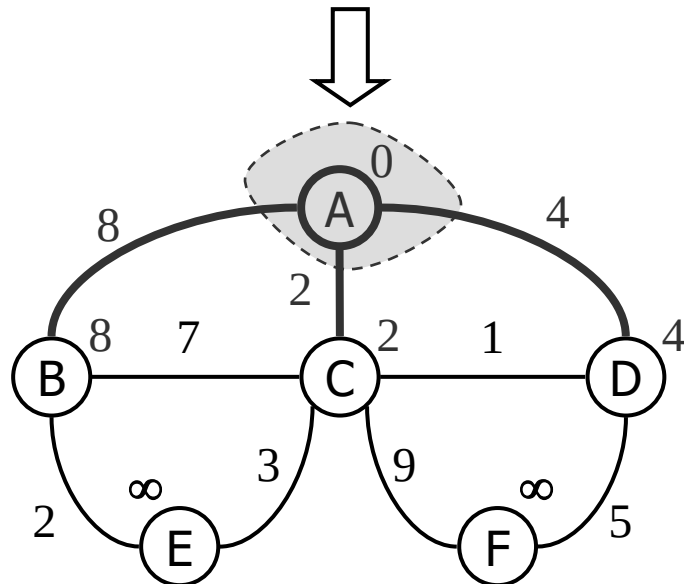
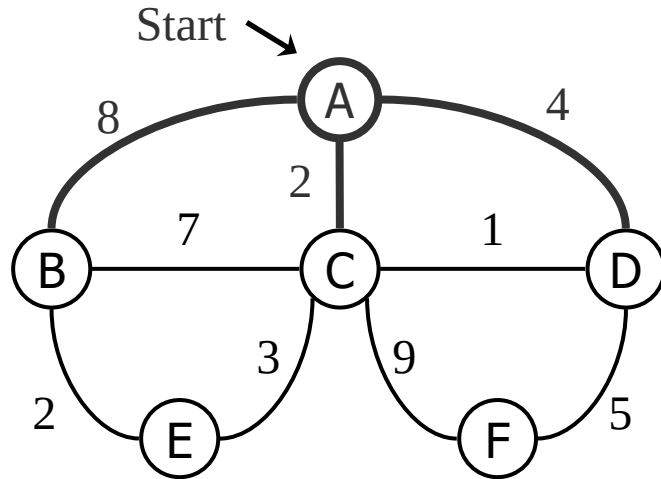
- ◆ To remove cycle
- ◆ Consider an edge $e = (u, z)$ such that
 - u is the vertex most recently added to the cloud
 - z is not in the cloud



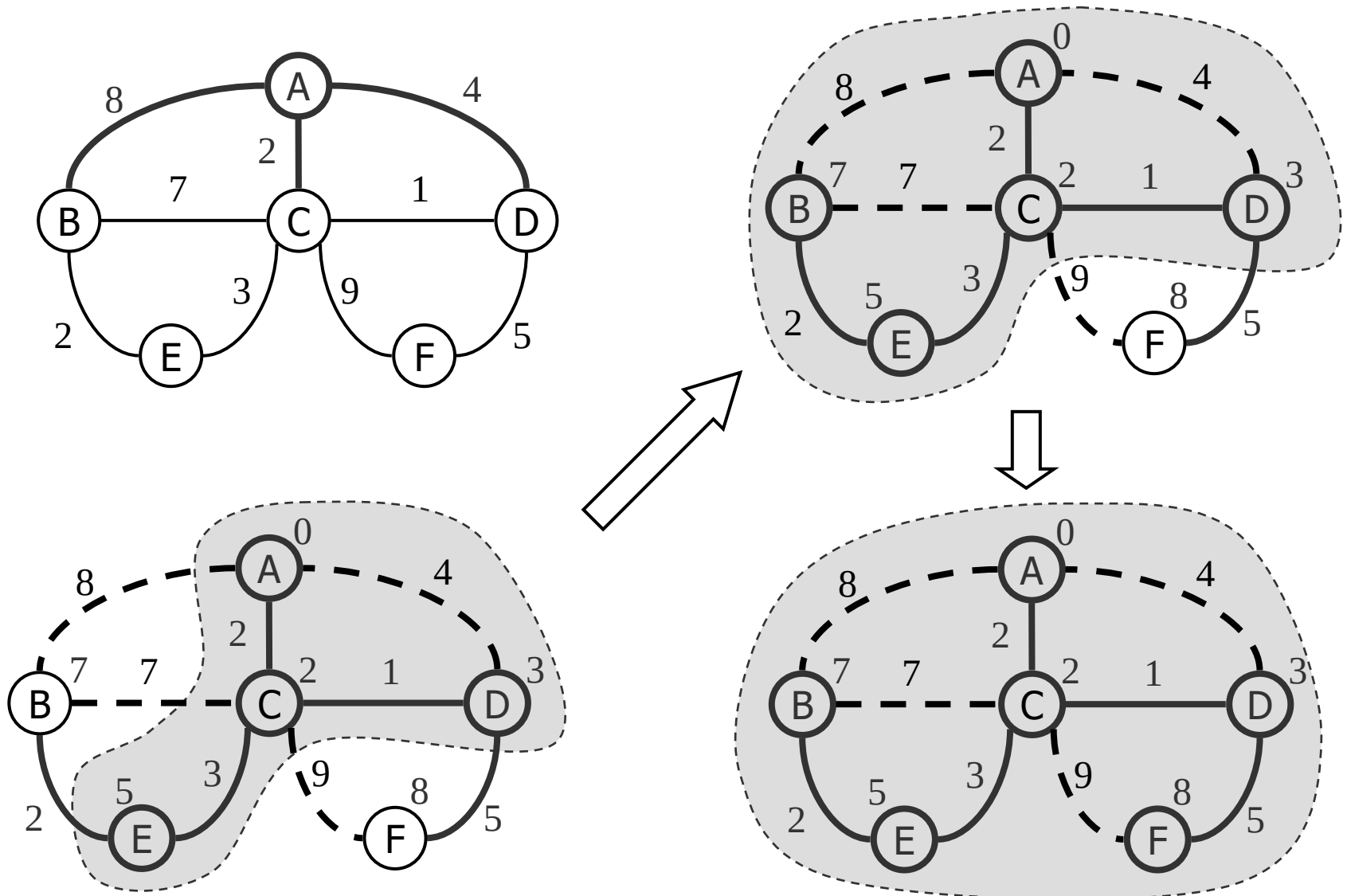
- ◆ The relaxation of edge e updates distance $d(z)$ as follows:
$$d(z) \leftarrow \min\{d(z), d(u) + \text{weight}(e)\}$$



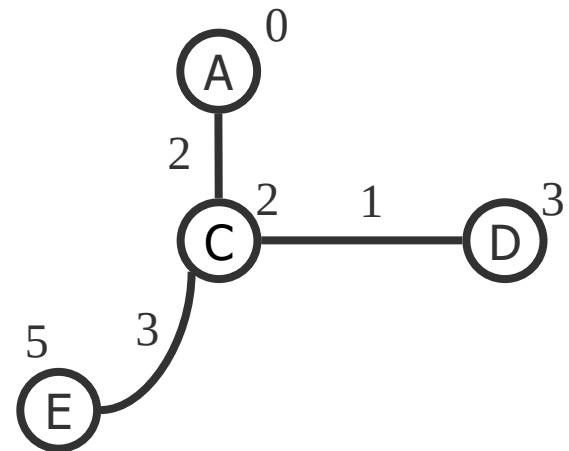
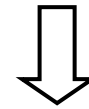
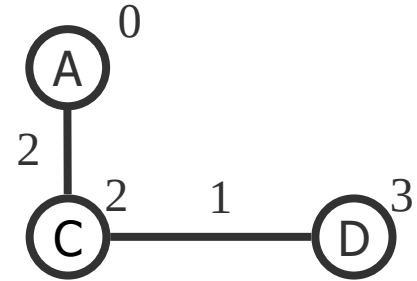
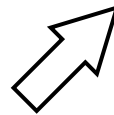
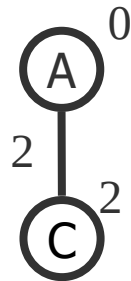
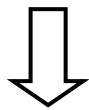
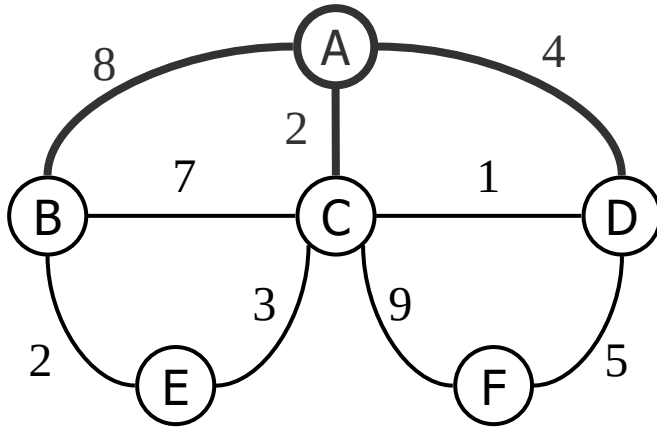
Dijkstra's - Detailed Steps



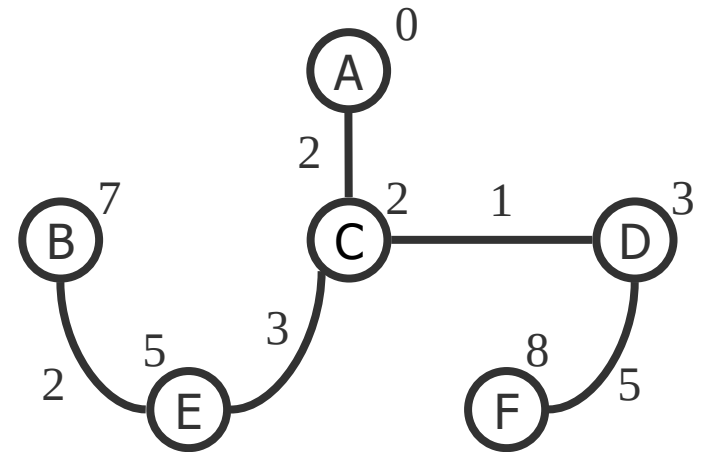
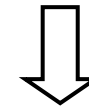
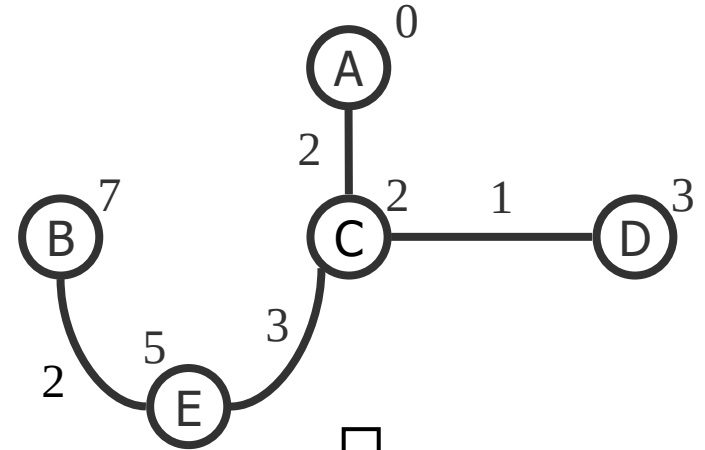
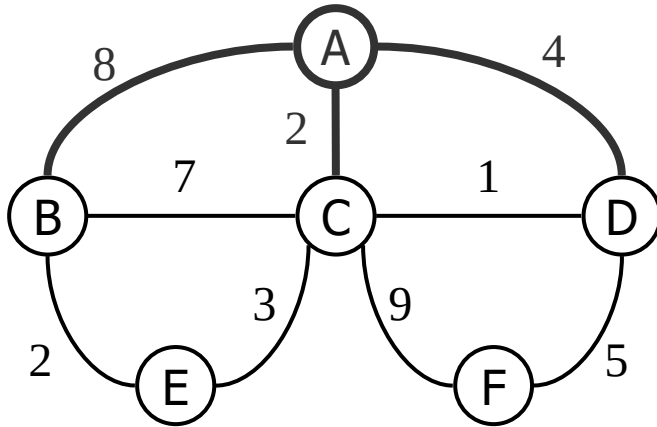
Dijkstra's - Detailed Steps (cont.)



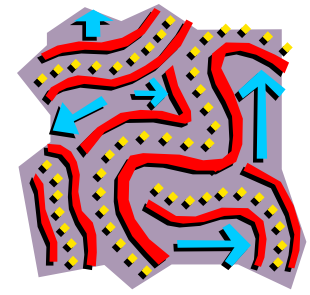
Dijkstra's - Simple Steps



Dijkstra's - Simple Steps (cont.)



Dijkstra's Algorithm



- ◆ A priority queue stores the vertices outside the cloud
 - Key: distance
 - Element: vertex
 - *insert(v)* inserts vertex *v* with distance as the key
- ◆ We store one label with each vertex:
 - Distance (*d(v)* label)

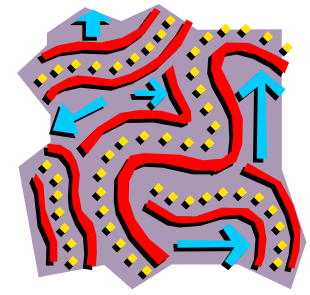
Algorithm DijkstraDistances(*G*, *s*)

Input: Graph *G* and vertex *s*

Output: Shortest paths from *s*

```
Q = empty minheap priority queue
  with distance as key
for each v in G.vertices()
  if v == s
    v.distance = 0
  else
    v.distance = ∞
  Q.insert(v)
while not Q.isEmpty()
  u = Q.removeMin()
  for each e in G.incidentEdges(u)
    // relax edge e
    z = G.opposite(u,e)
    r = u.distance + e.weight
    if r < z.distance
      z.distance = r
```

Analysis

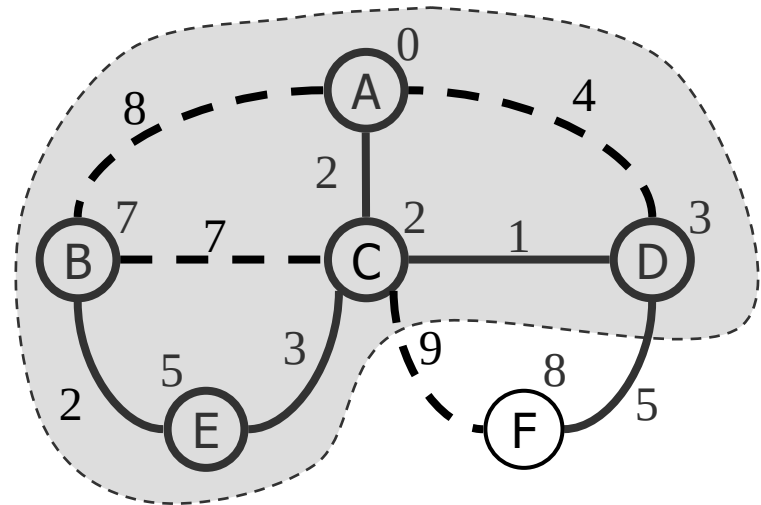


- ◆ Graph operations
 - Method `incidentEdges` is called once for each vertex
- ◆ Label operations
 - We set/get the distance of vertex z $O(\deg(z))$ times
 - Setting/getting a label takes $O(1)$ time
- ◆ Priority queue operations
 - Each vertex is inserted once into and removed once from the priority queue, where each insertion or removal takes $O(\log n)$ time
 - The key of a vertex in the priority queue is modified at most $\deg(w)$ times, where each key change takes $O(\log n)$ time
- ◆ Dijkstra's algorithm runs in $O((n + m) \log n)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$
- ◆ The running time can also be expressed as $O(m \log n)$ since the graph is connected

Why Dijkstra's Algorithm Works



- ◆ Dijkstra's algorithm is based on the greedy method. It adds vertices by increasing distance.
- Suppose it didn't find all shortest distances. Let F be the first wrong vertex the algorithm processed.
- When the previous node, D, on the true shortest path was considered, its distance was correct.
- But the edge (D,F) was **relaxed** at that time!
- Thus, so long as $d(F) \geq d(D)$, F's distance cannot be wrong. That is, there is no wrong vertex.

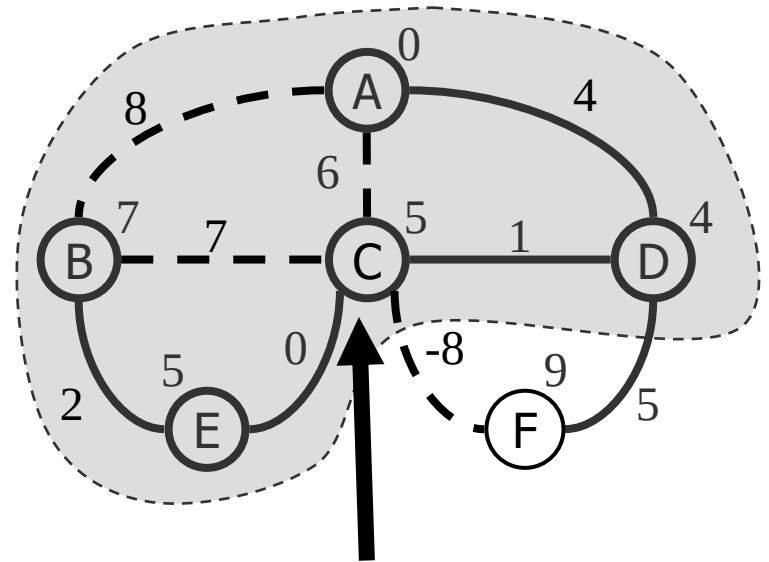


Why It Doesn't Work for Negative-Weight Edges



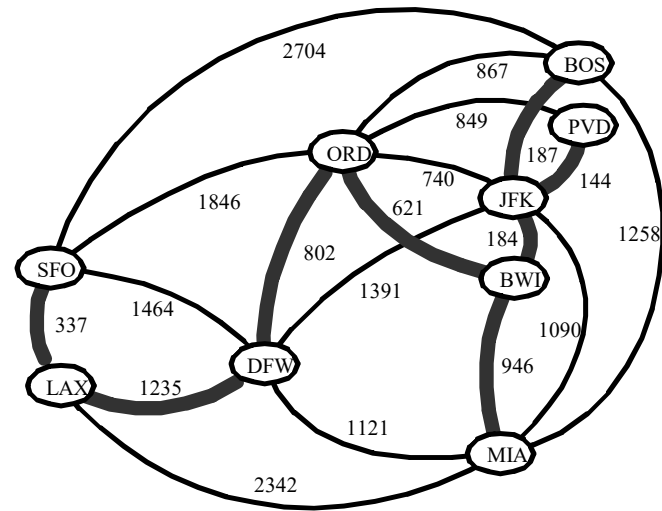
- ◆ Dijkstra's algorithm is based on the greedy method. It adds vertices by increasing distance.

- If a node with a negative incident edge were to be added late to the cloud, it could mess up distances for vertices already in the cloud.



C's true distance is 1, but it is already in the cloud with $d(C)=5$!

Minimum Spanning Trees (MST)



Outline and Reading

- ◆ Minimum Spanning Trees (7.3)
 - Definitions
 - A crucial fact
- ◆ The Prim-Jarnik Algorithm (7.3.2)
- ◆ Kruskal's Algorithm (7.3.1)



Minimum Spanning Tree

Spanning subgraph

- Subgraph of a graph G containing all the vertices of G

Spanning tree

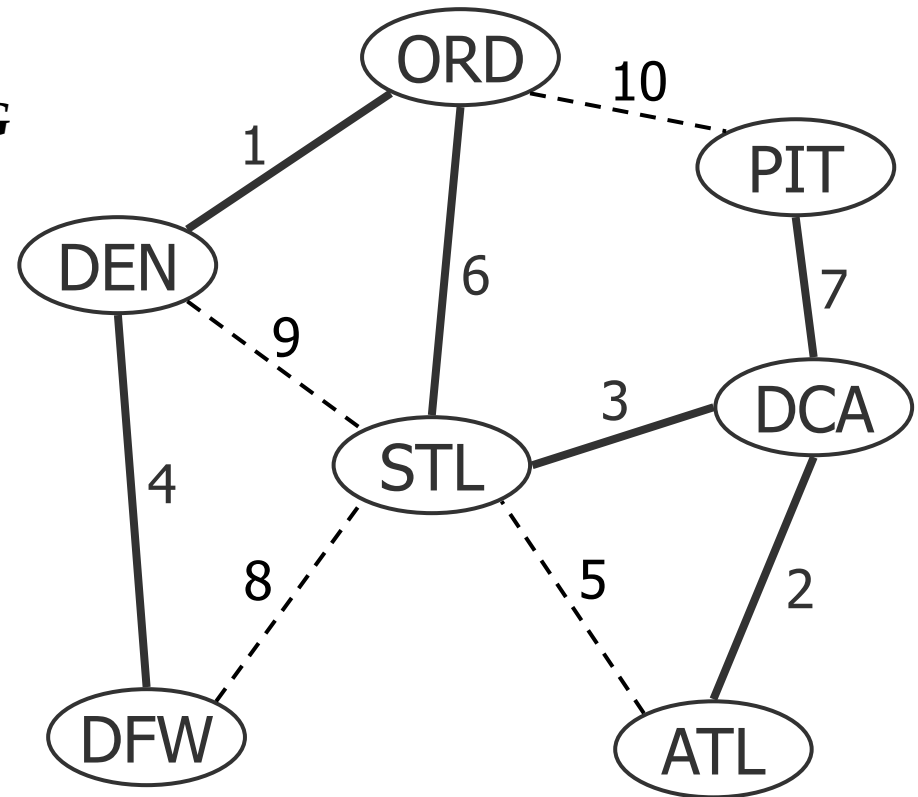
- Spanning subgraph that is itself a (free) tree

Minimum spanning tree (MST)

- Spanning tree of a weighted graph with minimum total edge weight

◆ Applications

- Communications networks
- Transportation networks



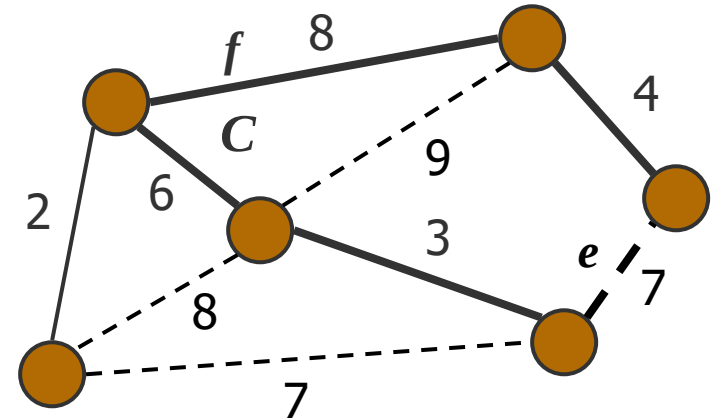
Cycle Property

Cycle Property:

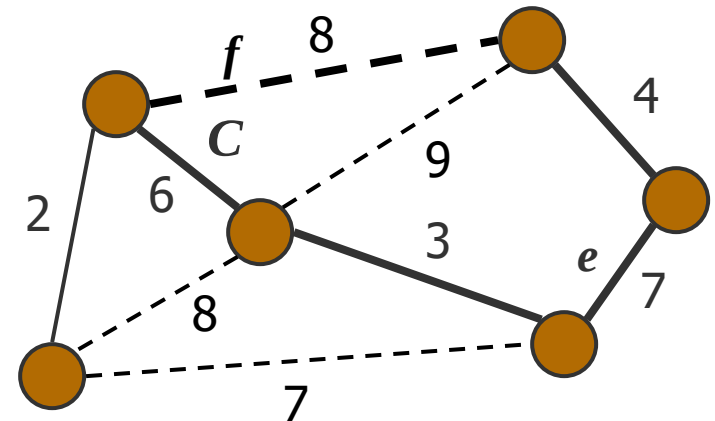
- Let T be a minimum spanning tree of a weighted graph G
- Let e be an edge of G that is not in T and C let be the cycle formed by e with T
- For every edge f of C , $\text{weight}(f) \leq \text{weight}(e)$

Proof:

- By contradiction
- If $\text{weight}(f) > \text{weight}(e)$ we can get a spanning tree of smaller weight by replacing e with f



Replacing f with e yields a better spanning tree



Partition Property

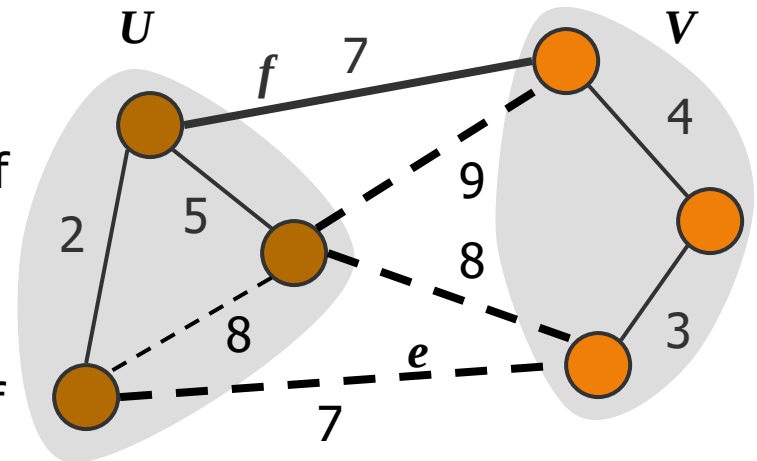
Partition Property:

- Consider a partition of the vertices of G into subsets U and V
- Let e be an edge of minimum weight across the partition
- There is a minimum spanning tree of G containing edge e

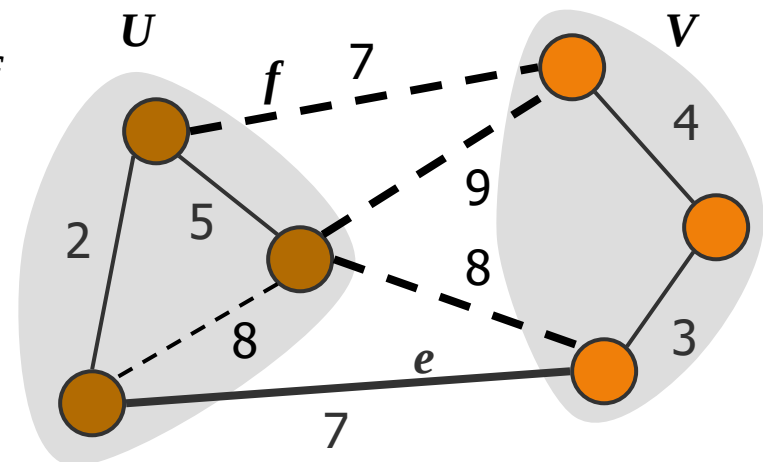
Proof:

- Let T be an MST of G
- If T does not contain e , consider the cycle C formed by e with T and let f be an edge of C across the partition
- By the cycle property,

$$\text{weight}(f) \leq \text{weight}(e)$$
- Thus, $\text{weight}(f) = \text{weight}(e)$
- We obtain another MST by replacing f with e

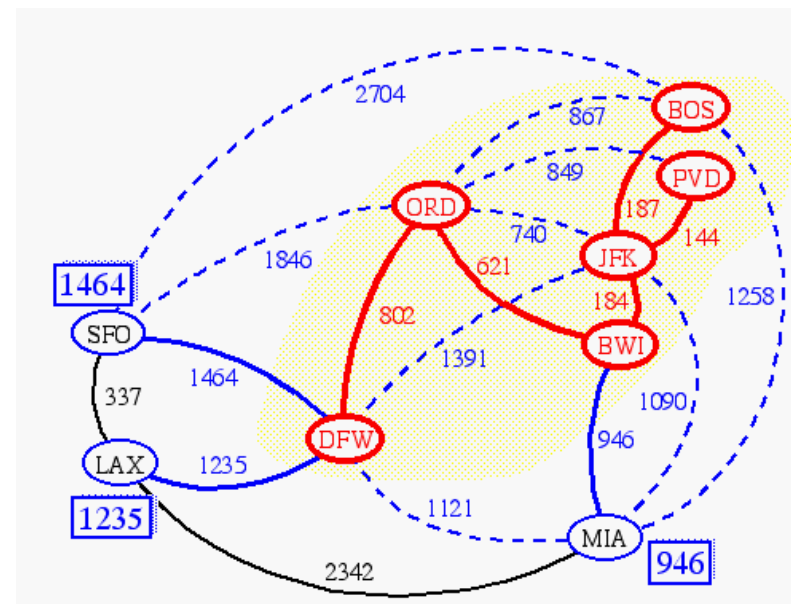


Replacing f with e yields another MST



Prim-Jarnik's Algorithm

- ◆ Similar to Dijkstra's algorithm (for a connected graph)
- ◆ We pick an arbitrary vertex s and we grow the MST as a cloud of vertices, starting from s
- ◆ We store with each vertex v a label $d(v)$ = the smallest weight of an edge connecting v to a vertex in the cloud
- ◆ At each step:
 - We add to the cloud the vertex u outside the cloud with the smallest distance label
 - We update the label of the vertices adjacent to u



Prim-Jarnik's Algorithm (cont.)

- ◆ A priority queue stores the vertices outside the cloud
 - Key: distance
 - Element: vertex
 - *insert(v)* inserts vertex *v* with distance as the key
- ◆ We store two labels with each vertex:
 - Distance
 - Parent edge in MST

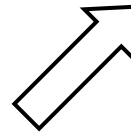
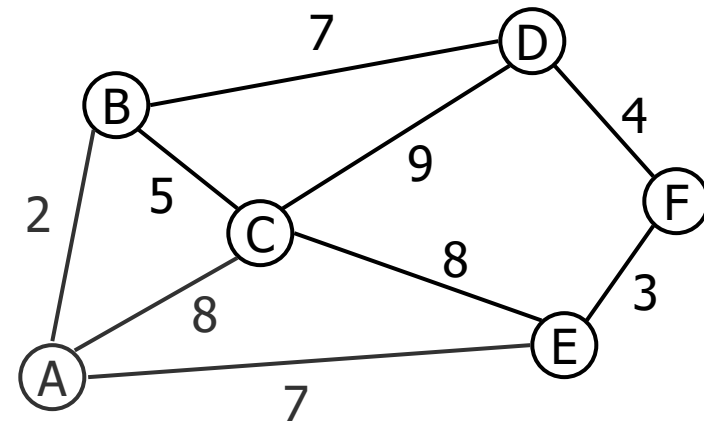
Algorithm PrimJarnikMST(*G*, *s*)

Input: Graph *G* and vertex *s*

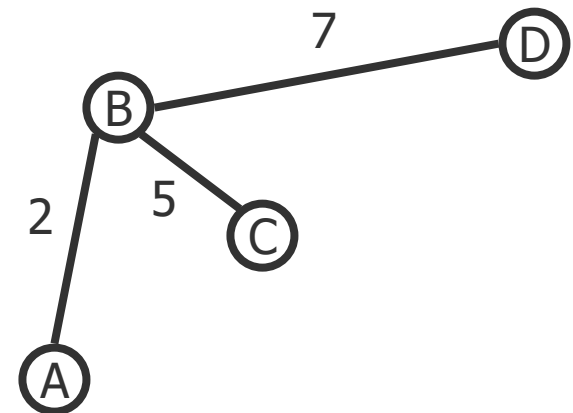
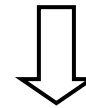
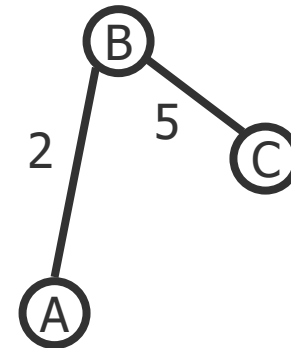
Output: An MST of *G*

```
Q = empty minheap priority queue
  with distance as key
for each v in G.vertices()
  if v == s
    v.distance = 0
  else
    v.distance = ∞
    v.parent = null
    Q.insert(v)
while not Q.isEmpty()
  u = Q.removeMin()
  for each e in G.incidentEdges(u)
    z = G.opposite(u,e)
    r = e.weight
    if r < z.distance
      z.distance = r
      z.parent = e
```

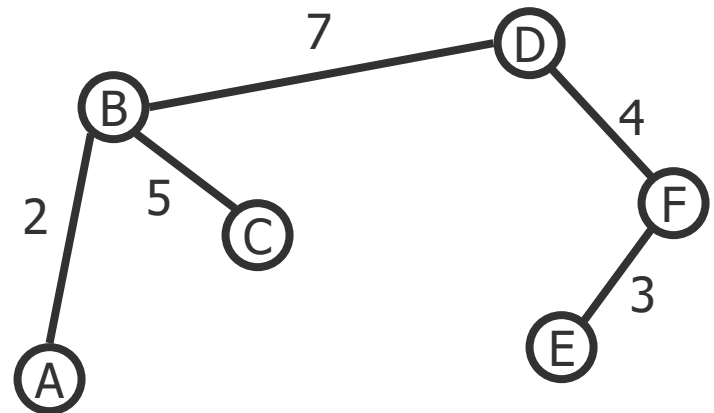
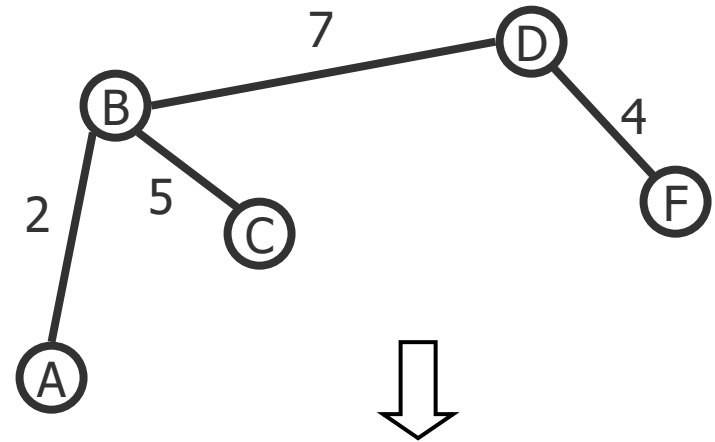
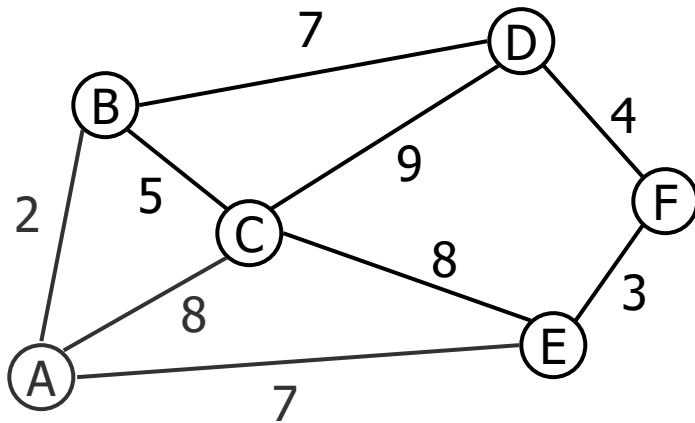
Prim-Jarnik Example



Start



Prim-Jarnik Example (cont.)



MST weight = 21

Analysis

- ◆ Graph operations
 - Method `incidentEdges` is called once for each vertex
- ◆ Label operations
 - We set/get the distance and parent labels of vertex z $O(\deg(z))$ times
 - Setting/getting a label takes $O(1)$ time
- ◆ Priority queue operations
 - Each vertex is inserted once into and removed once from the priority queue, where each insertion or removal takes $O(\log n)$ time
 - The key of a vertex w in the priority queue is modified at most $\deg(w)$ times, where each key change takes $O(\log n)$ time
- ◆ Prim-Jarnik's algorithm runs in $O((n + m) \log n)$ time provided the graph is represented by the adjacency list structure
 - Recall that $\sum_v \deg(v) = 2m$
- ◆ The running time is $O(m \log n)$ since the graph is connected

Kruskal's Algorithm

- ◆ A priority queue stores the edges outside the cloud
 - Key: weight
 - Element: edge
- ◆ At the end of the algorithm
 - We are left with one cloud that encompasses the MST
 - A tree T which is our MST

Algorithm KruskalMST(G)

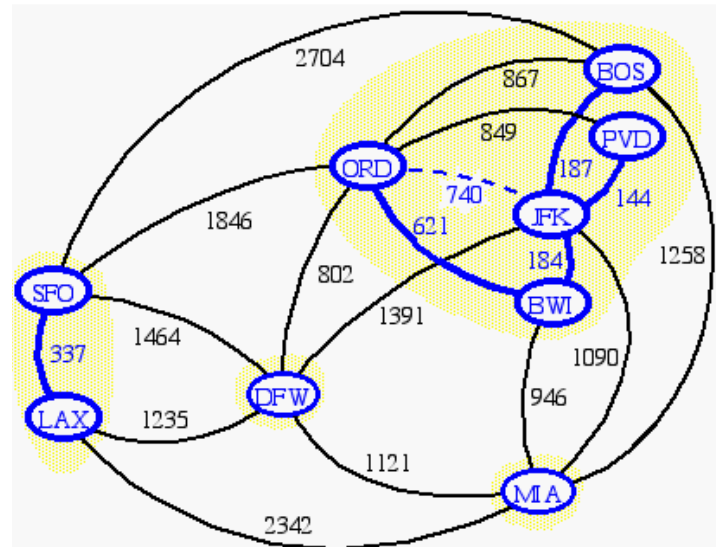
Input: A weighted graph G .

Output: An MST T for G .

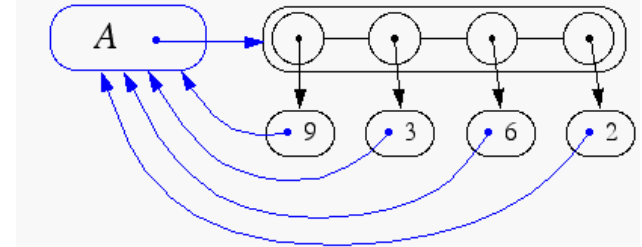
```
for each  $v$  in  $G$ 
    Cloud( $v$ ) = { $v$ }
 $Q$  = empty minheap priority queue with
    weight of edge as key
for each  $e$  in  $G$ .edges()
     $Q$ .add ( $e$ .weight,  $e$ )
 $T$  = empty tree
while  $T$ .edges() <  $n-1$ 
    edge  $e$  =  $Q$ .removeMin()
     $u, v$  = endpoints of  $e$ 
    if Cloud( $v$ ) != Cloud( $u$ )
         $T$ .addEdge( $e$ )
        Merge Cloud( $v$ ) and Cloud( $u$ )
return  $T$ 
```

Data Structure for Kruskal Algorithm

- ◆ The algorithm maintains a forest of trees
- ◆ An edge is accepted if it connects distinct trees
- ◆ We need a data structure that maintains a **partition**, i.e., a collection of disjoint sets, with the operations:
 - find**(u): return the set storing u
 - union**(u,v): replace the sets storing u and v with their union



Representation of a Partition



- ◆ Each set is stored in a sequence
- ◆ Each element has a reference back to the set
 - operation **find**(u) takes $O(1)$ time, and returns the set of which u is a member.
 - in operation **union**(u, v), we move the elements of the smaller set to the sequence of the larger set and update their references
 - the time for operation **union**(u, v) is $\min(n_u, n_v)$, where n_u and n_v are the sizes of the sets storing u and v
- ◆ Whenever an element is processed, it goes into a set of size at least double, hence each element is processed at most $\log n$ times

Partition-Based Implementation

- ◆ A partition-based version of Kruskal's Algorithm performs cloud merges as unions and tests as finds.

Algorithm Kruskal(G)

Input: A weighted graph G .

Output: An MST T for G .

P = partition of the vertices of G , where each vertex forms a separate set.

Q = empty minheap priority queue storing the edges of G , sorted by weights

T = empty tree

while not $Q.isEmpty()$

$(u,v) = Q.removeMin()$

 if $P.find(u) \neq P.find(v)$

$T.add(u,v)$

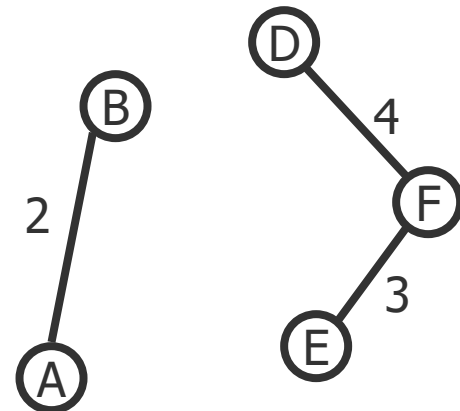
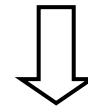
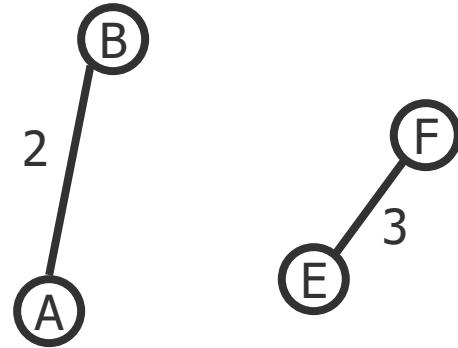
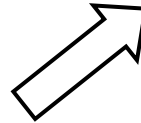
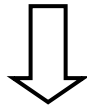
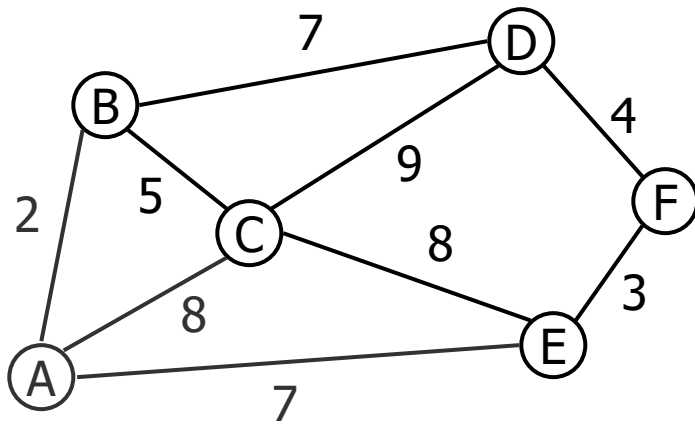
$P.union(u,v)$

return T

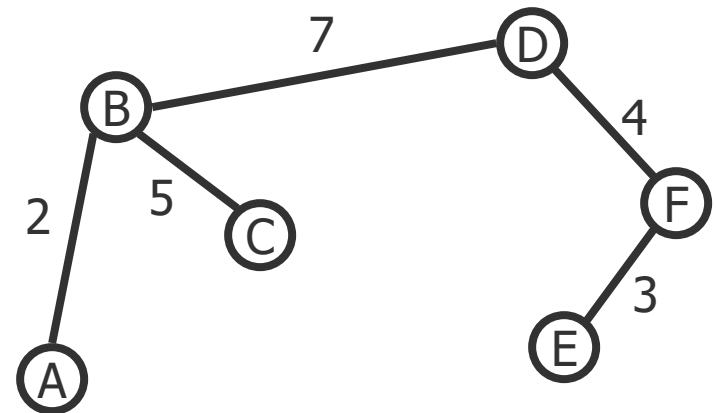
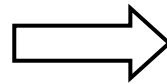
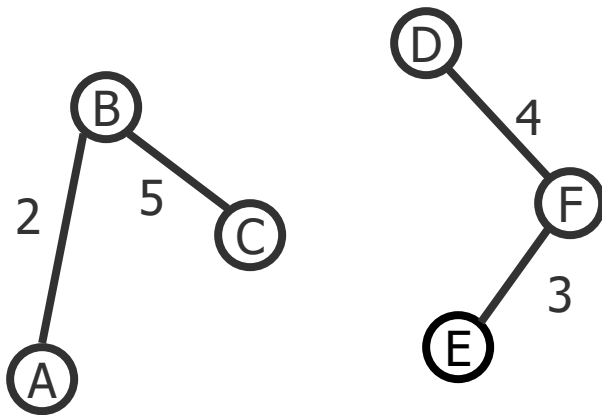
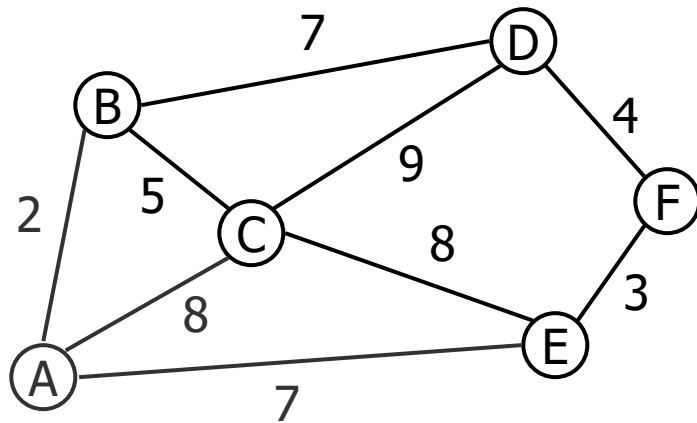
Running time:

$O(m \log n)$

Kruskal Example



Kruskal Example 1 – Simplest Diagram (cont.)



MST weight = 21